Python 3.12 Asynchronous Concurrency

Presented By Dr. Samriddhi Khade

DPU Course Material

Dr. D. Y. Patil Vidyapeeth, Pune (Deemed to be University)

CONFIDENTIAL ⊡ FOR ACADEMIC USE ONLY

Structured Concurrency with TaskGroup

- • Clean exception aggregation and auto-cancellation
- • Deterministic lifecycle without lingering coroutines
- • Zero dangling background task leaks

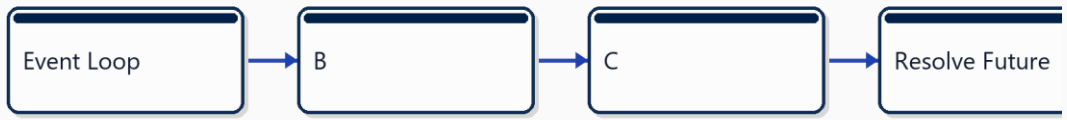
```python
                    // PYTHON
import asyncio

async def fetch_service(svc_id: int) -> str:
    await asyncio.sleep(0.05)
    return f"Service {svc_id} ready"

async def main():
    async with asyncio.TaskGroup() as tg:
        t1 = tg.create_task(fetch_service(1))
        t2 = tg.create_task(fetch_service(2))
    return t1.result(), t2.result()
```

AsyncIO Event Loop Workflow

- • Single-threaded cooperative task scheduling
- • Epoll event notification on OS sockets
- • Future resolution wakes paused coroutines



Best Practices in AsyncIO Systems

- • Never invoke blocking synchronous code on event loop
- • Delegate CPU intensive workloads to process pools
- • Utilize asyncio.timeout for network deadlines
- • Always await graceful cancellation on shutdown
- *Exercise Reference: exercise_2_async_gather*